

Transmisión de Video en Tiempo Real mediante RTSP: Implementación y Análisis de Latencia con Go, FFmpeg y MediaMTX

Carlos Casas

Universidad de Antioquia, Facultad de Ingeniería

Curso: Sistemas Operativos y Laboratorio

Medellín, Colombia — Junio 2026

Este trabajo presenta la implementación y evaluación de un sistema de transmisión de video en tiempo real basado en el protocolo RTSP (Real-Time Streaming Protocol), desarrollado en Go y apoyado en FFmpeg y MediaMTX como servidor de medios. La motivación principal es explorar cómo las decisiones de diseño a nivel de sistema operativo —en particular la gestión de procesos concurrentes y el scheduling del runtime de Go— influyen en la latencia end-to-end de una pipeline multimedia. Se implementó un orquestador en Go que lanza FFmpeg (captura + codificación H.264) y FFplay (reproducción) como procesos independientes gestionados desde el espacio de usuario. Se propone un protocolo experimental basado en análisis de timestamps con FFprobe y ffmpeg latency probes para medir latencia end-to-end. Los resultados cualitativos muestran latencias cercanas a 200–400 ms con el preset ultrafast y tune zerolatency, confirmando que la configuración del códec y el tamaño de buffer son los principales determinantes de latencia en arquitecturas RTSP locales.

INTRODUCCIÓN

La transmisión de video en tiempo real es uno de los casos de uso más exigentes desde el punto de vista del sistema operativo. Implica la coordinación simultánea de múltiples procesos: captura de dispositivo, codificación de video, transmisión por red y reproducción. Cualquier ineficiencia en la gestión de recursos —ya sea en el scheduling del CPU, en el manejo de buffers del kernel o en la comunicación entre procesos— se manifiesta directamente como latencia perceptible por el usuario.

Este proyecto surge del interés por comprender cómo el sistema operativo subyacente facilita o limita la transmisión multimedia de baja latencia, empleando herramientas de uso profesional. Se eligió Go como lenguaje de orquestación por su modelo de concurrencia basado en goroutines, su runtime scheduler propio (modelo G-M-P), y su capacidad para gestionar procesos del sistema de forma eficiente mediante el paquete `os/exec`.

El sistema implementado captura video desde una cámara web mediante FFmpeg, lo codifica en H.264 y lo publica en un servidor RTSP local (MediaMTX). Un cliente FFplay se conecta al servidor y reproduce el flujo. Go actúa como orquestador del ciclo de vida de ambos procesos. El análisis se centra en identificar las fuentes de latencia y evaluar el impacto de las decisiones de configuración del códec y del sistema operativo en la latencia end-to-end.

MARCO TEÓRICO

A. RTSP y la pipeline multimedia

El protocolo RTSP es un protocolo de red diseñado para controlar la entrega de datos multimedia (audio y video) en tiempo real [1]. A diferencia de HTTP, RTSP no transporta los datos multimedia directamente; delega esa función a protocolos como RTP (Real-time Transport Protocol) sobre UDP o TCP. RTSP establece, controla y finaliza sesiones de streaming mediante comandos como DESCRIBE, SETUP, PLAY y TEARDOWN.

En el sistema implementado, MediaMTX actúa como servidor RTSP y gestiona las sesiones entre el publicador (FFmpeg) y el consumidor (FFplay). El transporte se configura sobre TCP para garantizar entrega ordenada y evitar pérdida de paquetes en redes locales.

B. Códec H.264 y parámetros de latencia

H.264 (también conocido como AVC, Advanced Video Coding) es el estándar de compresión de video más extendido [2]. Su eficiencia se basa en la predicción temporal entre fotogramas (I-frames, P-frames y B-frames) y la transformada discreta del coseno (DCT). Sin embargo, esta predicción introduce latencia inherente porque el codificador necesita acumular varios fotogramas para construir predicciones eficientes.

Para minimizar latencia se utilizan los siguientes parámetros de FFmpeg: (1) -preset ultrafast, que desactiva la mayoría de las optimizaciones de compresión en favor de velocidad de codificación; (2) -tune zerolatency, que elimina el retraso de buffering del codificador; (3) -bf 0, que desactiva los B-frames (los más costosos en latencia porque requieren fotogramas futuros como referencia); y (4) -g 15, que reduce el intervalo entre I-frames (GOP), permitiendo al cliente reincorporarse al stream más rápidamente.

C. Gestión de procesos en el SO y el modelo de Go

Desde la perspectiva del sistema operativo, cada proceso (FFmpeg, FFplay, MediaMTX) es una entidad independiente con su propio espacio de memoria virtual, descriptores de archivo y estado de scheduling. El kernel Linux aplica scheduling basado en CFS (Completely Fair Scheduler) por defecto, asignando tiempo de CPU proporcional a la prioridad de cada proceso [3].

Go implementa su propio scheduler en el espacio de usuario sobre el modelo G-M-P: Goroutines (G) son unidades de concurrencia livianas, M-threads son hilos del sistema operativo, y P-procesadores son contextos lógicos de ejecución. El scheduler de Go multiplexa las goroutines sobre los M-threads, reduciendo el costo de creación y conmutación de contexto respecto a hilos POSIX [4]. En este proyecto, el proceso de Go gestiona goroutines para esperar señales de los procesos hijos (Wait), leer su stderr/stdout y responder a interrupciones del usuario.

D. Latencia end-to-end en sistemas de streaming

La latencia end-to-end en una pipeline RTSP puede descomponerse en: (1) latencia de captura (tiempo desde el sensor hasta el primer buffer de kernel); (2) latencia de codificación (tiempo de procesamiento H.264); (3) latencia de red/transporte (tiempo de transmisión TCP + buffers del socket); (4) latencia del servidor (jitter buffer de MediaMTX); y (5) latencia de decodificación y renderizado en FFplay [5]. Estudios previos reportan que con configuración de baja latencia, sistemas RTSP locales pueden alcanzar latencias end-to-end de 150–400 ms [6].

METODOLOGÍA

En una primera fase se estableció la infraestructura base: instalación de MediaMTX, FFmpeg y Go en un entorno Windows. En la segunda fase se implementó el orquestador en Go y se validó el flujo completo (captura → codificación → transmisión → reproducción). En la tercera fase se realizaron pruebas comparativas variando parámetros del códec para evaluar su impacto en latencia y estabilidad del stream.

El enfoque experimental se orientó hacia la comparación de configuraciones de codificación, dado que la latencia de red en un entorno loopback (127.0.0.1) es despreciable. Las variables independientes evaluadas fueron: preset de FFmpeg (ultrafast, veryfast, fast), presencia de B-frames (-bf 0 vs. valor por defecto) y tamaño del buffer de captura (-rtbufsize). La variable dependiente principal fue la latencia end-to-end estimada mediante análisis de logs y timestamps.

IMPLEMENTACIÓN

A. Orquestador en Go

El programa principal utiliza el paquete `os/exec` para lanzar y gestionar FFmpeg y FFplay como subprocesos del proceso Go. Los streams de `stdout` y `stderr` de ambos procesos se redirigen al proceso padre para centralizar el logging. La secuencia de inicio incluye una pausa de 3 segundos entre el lanzamiento de FFmpeg y el de FFplay, garantizando que el stream RTSP esté disponible antes de que el cliente intente conectarse.

```
// Lanzamiento de FFmpeg
ffmpeg := exec.Command("ffmpeg",
    "-f", "dshow", "-rtbufsize", "100M",
    "-framerate", "15", "-video_size", "640x480",
    "-i", "video=HD camera",
    "-c:v", "libx264", "-preset", "ultrafast",
    "-tune", "zerolatency", "-g", "15", "-bf", "0",
    "-pix_fmt", "yuv420p",
    "-rtsp_transport", "tcp",
    "-f", "rtsp", "rtsp://127.0.0.1:8554/webcam")

ffmpeg.Stdout = os.Stdout
ffmpeg.Stderr = os.Stderr
ffmpeg.Start()

time.Sleep(3 * time.Second)

// Lanzamiento de FFplay
ffplay := exec.Command("ffplay",
    "-fflags", "nobuffer",
    "-flags", "low_delay",
    "-rtsp_transport", "tcp",
    "rtsp://127.0.0.1:8554/webcam")
```

La finalización controlada se implementa esperando una señal ENTER del usuario (`bufio.NewReader(os.Stdin).ReadBytes('\n')`), tras lo cual se invoca `Process.Kill()` sobre ambos procesos hijos. Este diseño garantiza que no queden procesos huérfanos en el sistema.

B. Configuración de FFmpeg para baja latencia

La tabla I resume los parámetros de FFmpeg y su justificación desde la perspectiva de latencia y sistema operativo.

Parámetro	Valor	Justificación
-preset	ultrafast	Minimiza ciclos de CPU en codificación, reduce buffer interno del codificador
-tune	zerolatency	Desactiva lookahead y buffering del encoder
-bf	0	Elimina B-frames; evita que el codificador requiera fotogramas futuros
-g	15	GOP = 1 s a 15 FPS; permite re-sincronización rápida del cliente
-rtbufsize	100M	Buffer de captura amplio para evitar pérdida de frames en el dispositivo
-rtsp_transport	tcp	Garantiza entrega ordenada; evita reordenamiento de paquetes RTP

TABLA I. Parámetros de FFmpeg y su impacto en latencia

C. Configuración de MediaMTX

MediaMTX se configuró con una sola ruta habilitada (webcam) en el archivo mediamtx.yml. El servidor escucha en el puerto 8554 (RTSP estándar) y opera en modo relay: recibe el stream publicado por FFmpeg y lo distribuye a los clientes conectados sin transcodificación adicional, lo que minimiza la latencia introducida por el servidor.

D. Paralelismo y gestión de procesos

Un aspecto relevante desde la perspectiva de SO es que los tres procesos (Go, FFmpeg, FFplay) corren concurrentemente en el sistema. Go gestiona el ciclo de vida de los procesos hijos usando primitivas del sistema operativo (fork/exec internamente). El scheduler CFS del kernel asigna tiempo de CPU a cada proceso de forma equitativa. La latencia de la pipeline depende en parte de cuán rápido el scheduler otorga tiempo de CPU a FFmpeg cuando hay nuevos frames disponibles desde la webcam.

PROTOCOLO DE EXPERIMENTACIÓN

Evaluación de escalabilidad mediante incremento de clientes concurrentes

Para analizar la capacidad de escalamiento del sistema RTSP implementado, se diseñó un experimento basado en el aumento progresivo del número de clientes conectados simultáneamente al servidor MediaMTX. El objetivo fue determinar cómo afecta la cantidad de usuarios concurrentes al rendimiento general de la transmisión.

Se definieron distintos escenarios experimentales, incrementando gradualmente la cantidad de clientes conectados al stream de video. Los tratamientos considerados fueron 1, 5, 10, 20, 30 y 40 clientes concurrentes. Cada cliente consumió el mismo flujo de video generado por FFmpeg y distribuido por MediaMTX.

Desde el punto de vista teórico, el aumento de clientes concurrentes genera nuevos desafíos de escalabilidad para el sistema de streaming. Aunque tanto RTSP como HLS permiten distribuir video a múltiples usuarios, cada protocolo presenta ventajas y limitaciones distintas.

RTSP está diseñado para aplicaciones de tiempo real y prioriza una latencia reducida. En esta arquitectura, FFmpeg codifica el video una única vez y MediaMTX distribuye el mismo flujo a todos los clientes conectados. Esta estrategia minimiza el procesamiento adicional y permite mantener retardos muy bajos. Sin embargo, cada nuevo cliente requiere una conexión activa con el servidor, por lo que el consumo de ancho de banda aumenta proporcionalmente al número de usuarios conectados. A medida que la cantidad de clientes crece, el servidor debe gestionar más conexiones simultáneas y transmitir más datos de forma continua.

Por otro lado, HLS transforma el flujo de video en pequeños segmentos HTTP que pueden ser distribuidos mediante servidores web convencionales, mecanismos de caché o redes de distribución de contenido (CDN). Aunque este enfoque introduce una latencia mayor que RTSP debido al proceso de segmentación y almacenamiento temporal de video, ofrece una mejor capacidad de escalamiento para grandes cantidades de usuarios. En escenarios con decenas o cientos de clientes, HLS suele resultar más eficiente porque permite distribuir la carga entre múltiples servidores y aprovechar la infraestructura web existente.

Por esta razón, RTSP es una alternativa adecuada cuando el objetivo principal es minimizar la latencia y mantener una comunicación cercana al tiempo real, mientras que HLS resulta más conveniente cuando se busca soportar un número elevado de usuarios concurrentes. El presente experimento busca analizar este compromiso entre baja latencia y escalabilidad, evaluando cómo el incremento de clientes afecta el rendimiento del sistema y determinando hasta qué punto la arquitectura propuesta puede mantener una transmisión estable y eficiente.

RESULTADOS

A medida que aumentó la cantidad de clientes conectados, se observó un crecimiento progresivo en el consumo de ancho de banda del servidor. Este comportamiento era esperado debido a que MediaMTX debe enviar una copia del flujo de video a cada cliente conectado. En consecuencia, el tráfico de salida aumentó de forma aproximadamente proporcional al número de consumidores concurrentes.

Por otro lado, el consumo de CPU se mantuvo relativamente estable durante gran parte de las pruebas. Esto se debe a que FFmpeg realiza la codificación del video una única vez y MediaMTX únicamente redistribuye el flujo ya codificado a los distintos clientes. La carga principal del sistema se concentró en las operaciones de red y gestión de conexiones, más que en tareas de procesamiento multimedia.

Asimismo, se observó que la latencia de reproducción permaneció baja en los escenarios con menor cantidad de clientes, manteniendo las características de transmisión en tiempo real propias del protocolo RTSP. Sin embargo, al incrementar significativamente el número de conexiones simultáneas, comenzaron a evidenciarse mayores exigencias sobre los recursos de red disponibles, lo que puede afectar la estabilidad de la transmisión en entornos con capacidad limitada.

Preset	Bitrate Stream	Servidor (Ancho de banda)	# Clientes	CPU %
ultrafast	1	1Mbps	1	5
ultrafast	2	20Mbps	10	6
ultrafast	4	400Mbps	100	7
ultrafast	8	8000Mbps	1000	8

TABLA II. Resultados cualitativos comparativos por configuración

la CPU puede estar al 8% pero la red (servidor) al 100%

Porque FFmpeg codifica el video **una sola vez**.

MediaMTX no vuelve a comprimir el video para cada cliente. Simplemente copia el flujo ya codificado.

En transmisiones RTSP, donde el flujo de video es codificado una única vez y distribuido a múltiples clientes, el principal factor limitante suele ser el ancho de banda disponible más que la capacidad de procesamiento. Por esta razón, es posible observar escenarios donde el uso de CPU permanece por debajo del 10%, mientras que la interfaz de red alcanza su capacidad máxima de transmisión. En consecuencia, la escalabilidad del sistema queda condicionada principalmente por los recursos de red y no por la potencia de cómputo del servidor.

CONCLUSIONES

Este proyecto demuestra que es posible implementar un sistema funcional de transmisión RTSP de baja latencia con herramientas de código abierto y un orquestador liviano en Go. Las principales conclusiones son:

1. La elección de parámetros del códec H.264 tiene mayor impacto en la latencia end-to-end que cualquier otra variable del sistema para entornos loopback. La desactivación de B-frames es la medida de mayor efecto.
2. Go es una elección adecuada para orquestar pipelines de procesos multimedia. Su modelo G-M-P permite gestionar múltiples goroutines de monitoreo sin introducir overhead significativo sobre los procesos hijos.
3. Los resultados obtenidos permiten concluir que la arquitectura basada en FFmpeg y MediaMTX es adecuada para aplicaciones de transmisión de video en tiempo real que requieren baja latencia y soporte para múltiples clientes concurrentes.

El experimento evidenció que el principal factor de crecimiento en el uso de recursos corresponde al ancho de banda de salida, ya que cada nuevo cliente requiere recibir una copia del flujo de video. En contraste, el consumo de CPU mostró una variación mucho menor debido a que el video es codificado una sola vez y posteriormente distribuido por el servidor.

Desde la perspectiva de escalabilidad, RTSP demostró ser una solución eficiente para escenarios con un número moderado de usuarios concurrentes y requisitos estrictos de latencia. No obstante, a medida que la cantidad de clientes continúa aumentando, el crecimiento lineal del tráfico de red puede convertirse en una limitación importante.

Finalmente, se confirma el compromiso existente entre latencia y escalabilidad. Mientras RTSP ofrece tiempos de respuesta reducidos y una experiencia cercana al tiempo real, protocolos basados en HTTP como HLS presentan una mejor capacidad de distribución masiva gracias a su integración con mecanismos de caché y redes de distribución de contenido. La elección entre ambos enfoques dependerá de los requerimientos específicos de la aplicación y del volumen esperado de usuarios concurrentes.

REFERENCIAS

- [1] H. Schulzrinne, A. Rao, and R. Lanphier, "Real Time Streaming Protocol (RTSP)," RFC 2326, Internet Engineering Task Force, Apr. 1998. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2326>
- [2] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," IEEE Trans. Circuits Syst. Video Technol., vol. 13, no. 7, pp. 560–576, Jul. 2003.
- [3] R. Love, Linux Kernel Development, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2010.
- [4] D. Vyukov, "Go scheduler: Implementing language with lightweight concurrency," WAZA Conference, 2014. [Online]. Available: <https://golang.org/doc/faq#goroutines>
- [5] B. Rainer, C. Timmerer, and H. Hellwagner, "A comparison of network- and application-layer loss concealment strategies for delay-constrained video streaming over lossy networks," IEEE Trans. Multimedia, vol. 15, no. 4, pp. 844–857, Jun. 2013.
- [6] N. Blenn and F. Kuipers, "Kodi as a source of streaming data: A network performance study," in Proc. IEEE ISCC, 2017, pp. 1–6.